

Code Climb: Online Judge Platform

High-Level Design Document

Frontend React + Vite	Backend Node + Express	Database MongoDB Atlas	Execution Docker
---------------------------------	----------------------------------	----------------------------------	----------------------------

1. Overview

An Online Judge platform where users browse algorithmic problems, submit code, and receive automated verdicts. Version 2 adds a leaderboard an AI assistant for hints, submission analysis, and next-problem recommendations to create a personalized learning path.

Features

- Authentication (register / login)
- Problem list & problem statement page
- Online code editor with multi-language submission
- Automated verdict via Dockerized sandbox
- Leaderboard & user profile
- AI guidance: hints, WATLE report, problem recommendations

2. Technology Stack

Layer	Technology	Purpose
UI	React 18 + Vite 5 + Redux	SPA framework with fast build; React Router v6 for routing
Styling	Tailwind CSS v3	Utility-first CSS; rapid, consistent UI
HTTP Client	Axios	JWT injection via interceptors; promise-based requests
Backend	Node.js 20 + Express 4	REST API server; handles auth, problems, submissions
Auth	JWT + bcryptjs	Stateless tokens (with expiry); bcrypt password hashing
Database	MongoDB Atlas + Mongoose	Flexible schema for problems & submissions
Caching	Redis	Leaderboard cache; rate-limit submission requests
Code Execution	Docker (per-language containers)	Isolated sandbox with CPU/memory/time limits
AI	Google Gemini 1.5 Flash API	Hints, submission reports, and problem recommendations
CI/CD	GitHub Actions + Vercel/AWS	Automated tests, build, and deploy on merge

3. Database Design

MongoDB Atlas with four collections. Mongoose handles schema validation and relationships.

User

Field	Type	Notes
userID	String (UUID)	Primary key, auto-generated
email	String	Unique index
passwordHash	String	bcrypt hash; raw password never stored
fullName	String	Display name
score	Number	Updated on each accepted submission
noOfProblems	Number	Stores no. of problems solved

Problem

Field	Type	Notes
problemID	String	Slug-style, e.g. 'two-sum'
title	String	Display title
statement	String (Markdown)	Rendered client-side with react-markdown + KaTeX
difficulty	String	'Easy' 'Medium' 'Hard'
topics	[String]	Tags, e.g. ['Array', 'Hash Map']
timeLimitMs / memoryLimitMB	Number	Per-problem Docker execution constraints

TestCase

Field	Type	Notes
testCaseID	String	UUID
problemID	ObjectId (ref: Problem)	Foreign key; indexed
input expectedOutput	String	Whitespace-trimmed before comparison
isHidden	Boolean	Hidden cases not shown to user

Submission

Field	Type	Notes
submissionNo	Number	Auto-increment
problemID / userID	String (FK)	Indexed foreign keys
language	String	'cpp' 'python' 'java' 'javascript' 'go'

code	String(optional)	Raw source code
verdict	String	'AC' 'WA' 'TLE' 'MLE' 'RE' 'CE' 'Pending'
runtime / memory	Number	ms and KB of slowest passing test case
aiReport	String (optional)	Gemini-generated analysis; generated once on WA/TLE

4. Backend — REST API

Node.js 20 + Express 4. JWT authentication on all routes except register/login. Request bodies validated with Zod.

Method	Endpoint	Description
POST	/api/auth/register	Register user; hash password; return JWT
POST	/api/auth/login	Validate credentials; return access + refresh tokens
GET	/api/problems	Paginated list; filter by difficulty, topic, search
GET	/api/problems/:id	Full problem detail with sample test cases
POST	/api/submissions	Submit code; triggers Docker execution; returns submissionNo
GET	/api/submissions/:id	Poll for verdict (client polls every 2 s)
POST	/api/run	Run code with custom input (Playground); no verdict stored
GET	/api/leaderboard	Top users by score; Redis-cached for 60 s
GET	/api/users/:id	User profile and submission stats
POST	/api/ai/hint	Send problem + user code to Gemini; return tiered hint
POST	/api/ai/report	Send WA/TLE context to Gemini; return analysis report
POST	/api/ai/recommend	Send solve history to Gemini of past 10 problems; return 3 problem suggestions

Submission Flow

- Client POSTs code + language + problemID → Express validates JWT and creates Submission (verdict: Pending)
- Backend dispatches job to Docker execution service (async)
- Docker runs code against all test cases; first failure sets verdict (WA / TLE / MLE / RE)
- Verdict, runtime, memory written back; user score updated on AC
- Client polls GET /api/submissions/:id every 2 s until verdict is resolved

5. Code Execution — Docker

Each language runs in a dedicated container with no network access, read-only filesystem, and strict resource caps. Each submission adds the user code in a queue for execution to reduce concurrent requests.

Language	Base Image	Time Limit	Memory	Run Command
C++ 17	gcc:13-alpine	2 s	256 MB	g++ -O2 && ./a.out
Python 3.11	python:3.11-slim	5 s	256 MB	python3 solution.py
Java 21	eclipse-temurin:21-jre	5 s	512 MB	javac && java Main
JavaScript	node:20-alpine	5 s	256 MB	node solution.js

Security

- **--network none:** no internet access inside container
- **--read-only:** filesystem is read-only except /tmp
- **--cap-drop ALL:** all Linux capabilities dropped
- **--pids-limit 50:** prevents fork bombs
- **Verdict codes:** AC | WA | TLE | MLE | RE | CE

6. AI Integration — Gemini

All Gemini calls go through the backend only to protect the API key and enable caching and rate limiting.

Feature	How it works
Tiered Hints (L1/L2/L3)	User explicitly requests a hint. Prompt contains problem statement + user code. L1 = nudge, L2 = algorithmic direction, L3 = near-solution pseudocode. Verified by re-running suggested fix in Docker (up to 3 retries).
WA / TLE Report	Auto-triggered on wrong verdict. Gemini receives failing test case input, expected vs actual output, and user code. Returns issue summary, suggested fix, and complexity notes. Cached in aiReport field.
Next Problem Recommendation	Triggered from Profile. Sends last 10 solved problems (topic + difficulty). Gemini returns 3 problem IDs with reasoning. Backend validates IDs against database to prevent hallucinations.

Guardrails

- **On-demand only:** AI calls fire on explicit user action, never automatically
- **Redis caching:** Responses cached by (problemID + codeHash) to avoid duplicate Gemini calls
- **Model choice:** gemini-1.5-flash for speed and cost

Architecture Overview

